Assignment 3: Build your own pipeline

Python data processing pipeline: load, clean, add_features, and a pytest test

Name: Farzain Ahmed
Trainee ID: TRN-2026-001153
Course: Artificial Intelligence
Cohort: PSEB-AI-KHI-01 (PSEB-AI-KHI-02)
Training dates: Friday, 30 Oct 2026 to Thursday, 31 Dec 2026
Class mode: Online (weekend shift)
Lead trainer: Lajvanti Ratnani

1. Task

Create a simple data processing pipeline in Python: pipeline.py with load(), clean() and add_features(); run it on students_messy.csv and save students_clean.csv; write test_pipeline.py with one pytest test for clean(); run pytest. Check from the assignment: students_clean.csv has 5 rows and 5 columns, and pytest says 1 passed.

2. Files in this submission

- **pipeline.py**: the three functions and the code that runs them.
- **test_pipeline.py**: one pytest test for clean().
- **students_messy.csv**: the input data (7 rows, 4 columns: name, city, age, score).
- **students_clean.csv**: the generated output (5 rows, 5 columns).
- **Assignment3_Report.pdf**: this document (code, output, and screenshot pages).

3. How the pipeline works

- **load():** reads students_messy.csv with pandas.
- **clean():** fixes the name and city text (extra spaces and capital letters), drops the row with no name, turns the age written as a word ('twenty') into 20, removes the duplicate row, fills the missing score with the median score, and makes age and score whole numbers.
- **add_features():** adds a new column *passed* (True when the score is 50 or more).
- **main():** runs load, clean and add_features in order, then saves students_clean.csv.

Rows changed by clean(): 7 rows in, 5 rows out. Removed: the duplicate Omar row and the row with no name. Fixed: ' karachi' and 'KARACHI ' became 'Karachi', 'lahore' became 'Lahore', Omar's age 'twenty' became 20, and Omar's missing score was filled with 76 (the median score).

4. pipeline.py

```
# ============================================================
# Assignment 3: Build your own pipeline (load -> clean -> add_features)
# Name       : Farzain Ahmed
# Trainee ID : TRN-2026-001153
# Course     : Artificial Intelligence
# Cohort     : PSEB-AI-KHI-01 (PSEB-AI-KHI-02)
# Trainer    : Lajvanti Ratnani
# ============================================================
import pandas as pd

# number words, so that an age written as "twenty" becomes 20
UNITS = {"zero": 0, "one": 1, "two": 2, "three": 3, "four": 4, "five": 5, "six": 6,
         "seven": 7, "eight": 8, "nine": 9, "ten": 10, "eleven": 11, "twelve": 12,
         "thirteen": 13, "fourteen": 14, "fifteen": 15, "sixteen": 16,
         "seventeen": 17, "eighteen": 18, "nineteen": 19}
TENS = {"twenty": 20, "thirty": 30, "forty": 40, "fifty": 50, "sixty": 60,
        "seventy": 70, "eighty": 80, "ninety": 90}


def age_to_number(value):
    """Turn "21" or "twenty one" into a number; anything else becomes NaN."""
    if pd.isna(value):
        return float("nan")
    try:
        return float(value)
    except (TypeError, ValueError):
        pass
    total = 0
```

```
    for word in str(value).lower().replace("-", " ").split():
        if word in UNITS:
            total += UNITS[word]
        elif word in TENS:
            total += TENS[word]
        else:
            return float("nan")
    return float(total)


def load(path="students_messy.csv"):
    """Step 1: load the messy data from a CSV file."""
    return pd.read_csv(path)


def clean(df):
    """Step 2: clean the data (returns a new DataFrame)."""
    df = df.copy()

    # 1. fix text columns: remove extra spaces and use Title Case
    df["name"] = df["name"].astype("string").str.strip().str.title()
    df["city"] = df["city"].astype("string").str.strip().str.title()

    # 2. drop rows with a missing or empty name
    df = df.dropna(subset=["name"])
    df = df[df["name"] != ""]

    # 3. make age and score numeric ("twenty" -> 20)
    df["age"] = df["age"].apply(age_to_number)
    df["score"] = pd.to_numeric(df["score"], errors="coerce")

    # 4. remove duplicate rows
    df = df.drop_duplicates()

    # 5. fill missing age and score with the median value
    df["age"] = df["age"].fillna(df["age"].median())
    df["score"] = df["score"].fillna(df["score"].median())

    # 6. use whole numbers for age and score
    df["age"] = df["age"].astype(int)
    df["score"] = df["score"].astype(int)

    return df.reset_index(drop=True)


def add_features(df):
    """Step 3: add a new feature: passed (True when the score is 50 or more)."""
    df = df.copy()
    df["passed"] = df["score"] >= 50
    return df


def main():
    print("Name: Farzain Ahmed | Trainee ID: TRN-2026-001153")
    print("-" * 55)

    df = load("students_messy.csv")
    print(f"Loaded students_messy.csv: {df.shape[0]} rows, {df.shape[1]} columns")

    df = clean(df)
    print(f"After clean():             {df.shape[0]} rows, {df.shape[1]} columns")

    df = add_features(df)
    print(f"After add_features():      {df.shape[0]} rows, {df.shape[1]} columns")

    df.to_csv("students_clean.csv", index=False)
    print("Saved students_clean.csv\n")
    print(df.to_string(index=False))


if __name__ == "__main__":
    main()
```

5. test_pipeline.py

```
# Assignment 3: pytest test for clean()
# Name: Farzain Ahmed | Trainee ID: TRN-2026-001153
import pandas as pd
from pipeline import clean
```

```
def test_clean():
    messy = pd.DataFrame({
        "name": ["Ali", "Sara", "Omar", "Omar", None],
        "city": ["Karachi", " karachi", "KARACHI ", "KARACHI ", "Lahore"],
        "age": ["21", "22", "twenty", "twenty", "23"],
        "score": [80.0, 92.0, None, None, 66.0],
    })

    result = clean(messy)

    # the duplicate Omar row and the row with no name are removed
    assert result["name"].tolist() == ["Ali", "Sara", "Omar"]
    # spaces and capital letters in city are fixed
    assert result["city"].tolist() == ["Karachi", "Karachi", "Karachi"]
    # "twenty" becomes 20
    assert result["age"].tolist() == [21, 22, 20]
    # Omar's missing score is filled with the median of 80 and 92
    assert result["score"].tolist() == [80, 92, 86]
    # no missing values are left
    assert result.isna().sum().sum() == 0
```

6. Data

Input: students_messy.csv (7 rows, 4 columns)

```
name,city,age,score
Ali,Karachi,21,80.0
Sara, karachi,22,92.0
Omar,KARACHI ,twenty,
Omar,KARACHI ,twenty,
Hina,Lahore,20,72.0
,Lahore,23,66.0
Bilal,lahore,22,38.0
```

Output: students_clean.csv (5 rows, 5 columns)

```
name,city,age,score,passed
Ali,Karachi,21,80,True
Sara,Karachi,22,92,True
Omar,Karachi,20,76,True
Hina,Lahore,20,72,True
Bilal,Lahore,22,38,False
```

7. Running the pipeline

Command: `python pipeline.py`. Output when the code was run:

```
Name: Farzain Ahmed | Trainee ID: TRN-2026-001153
------------------------------------------------------
Loaded students_messy.csv: 7 rows, 4 columns
After clean():             5 rows, 4 columns
After add_features():      5 rows, 5 columns
Saved students_clean.csv

  name    city  age  score  passed
   Ali Karachi   21     80    True
  Sara Karachi   22     92    True
  Omar Karachi   20     76    True
  Hina  Lahore   20     72    True
 Bilal  Lahore   22     38   False
```

(Paste your screenshot here: terminal after running python pipeline.py)

8. Running pytest

Command: `pytest`. Expected result:

```
test_pipeline.py .                                                   [100%]
============================== 1 passed ==============================
```

(Paste your screenshot here: terminal showing the pytest result: 1 passed)

9. Conclusion

The pipeline loads the messy student data, cleans it, adds the passed feature, and saves students_clean.csv with 5 rows and 5 columns. The pytest test checks that clean() fixes the text, converts 'twenty' to 20, removes the duplicate and the row with no name, fills the missing score, and leaves no missing values.